

Proving the Program You Run

Mutable borrows inside a dependent type theory, by observation and closing off

Abstract. Verified systems code today is written twice: once as the efficient imperative program that runs, and once as a pure model that the proofs are about, joined by a refinement proof or a translation. We present Ochr, a core calculus in which the two are the same text. Ochr combines Rust-style mutable borrows with a Lean-style dependent type theory, and its definitional equality unfolds in-place code exactly as Lean unfolds pure code. Three ideas make this work. First, an *observational* equality `Id A t u` compares two computations by what they return and what they leave behind in the places they may write; it is not a new primitive but computes to ordinary equality between observations. Second, a call whose body is stuck on an unknown value is *closed off*: its result, and the final content of every place it borrowed, become *sealed programs*, closed source programs that own their state. Sealed programs are the backward functions of Aeneas, but written in the source language, so the programmer never sees a second language. Third, induction hypotheses are typed in the caller’s environment, where the borrow structure around the recursive argument performs the congruence step that a pure proof would write by hand. Proofs about in-place functions become plain structural recursion, frequently shorter than the corresponding proofs about pure functions. We give the calculus, its worked examples (including a function that returns a borrow into its argument), a translation into the calculus of inductive constructions that establishes consistency, and an executable Lean implementation of the checker.

1 Introduction

Here is an addition on Peano numerals that allocates nothing. It walks down the successors of `*x` until it reaches the final `Z` and moves `y` into that spot:

```
AddM : Π(x : &Nat) (y : Nat). Unit
AddM x y := match *x { Z => *x := y | S p => AddM &p y }
```

`&Nat` is a mutable borrow in the sense of Rust: a unique, temporary right to read and write a `Nat` owned by someone else. In the successor case the pattern variable `p` names the predecessor field in place, and `&p` reborrows it for the recursive call. Here is a theorem about `AddM`, stated and proved in the same language:

```
AddMZero : Π(x : &Nat). Id Unit (AddM x 0) ()
AddMZero x := match *x { Z => refl | S p => AddMZero &p }
```

The statement says that running `AddM x 0` is indistinguishable from doing nothing: both return `()`, and both leave the number behind `x` as it was. The proof is structural recursion and nothing else. There is no pure model of `AddM`, no refinement relation, no loop invariant, no separation-logic assertion and no translation into another language. The checker accepts the recursive call as a proof of the successor case by running both sides of the equation on a symbolic input and comparing what they leave behind.

This paper is about the type theory that makes that possible.

1.1 The two-language problem

Verified software is usually built in one of two ways. In the first, everything is written in a pure proof assistant such as Lean or Coq, and performance comes from the compiler’s ability to turn functional updates into in-place ones where it can prove the old value is dead. In the second, the efficient program is written in an imperative language, and a separate pure specification is related to it by a refinement proof: Verus [9] and Creusot [5] check Rust against specifications discharged by SMT, VeriFast checks C against separation-logic contracts, and Aeneas [7] translates safe Rust into a pure Lean program about which the user then proves theorems.

Both routes keep two artifacts. In the second route this is visible: there is a program and there is a specification, and the proof is about their relationship. In the first route it is hidden but present: the program the proof is about is not the program that runs, and the gap is bridged by an optimisation the programmer cannot state or rely on.

Aeneas comes closest to closing the gap. Its insight is that Rust’s ownership discipline makes mutation equivalent to state passing: a function taking $x : \&\text{mut } T$ is a pure function returning, besides its result, a *backward function* that computes the final value of $*x$. Aeneas performs this translation once, eagerly, for a whole program, and the user proves theorems about the translated Lean code, where the backward functions appear as ordinary definitions. The user therefore works in two languages at once, and must understand how one maps onto the other. We show that this translation can be performed **inside the type checker, lazily, as definitional equality, and in the source language itself.**

1.2 Three ideas

Observational equality. The type $\text{Id } A \ t \ u$ compares two computations t and u of result type A , run from the current state. It holds when they return equal values and leave equal contents in every place they may write. Id is not a new primitive with its own introduction and elimination rules. At a given environment it computes to an ordinary propositional equality between two *observations*, the tuples of result and final contents, and that equality computes further by the structure of the data in the manner of observational type theory [3]. The places a computation may write are read off its syntax, so they are an output of the computation like its result. This is the sense in which Ochr has no pure/impure divide: any program may appear in a type, and what it does to its environment is part of what the type talks about.

Closing off. Definitional equality in Ochr is normalisation by a deterministic symbolic machine: two terms are equal when the machine sends them to the same normal form. On symbolic inputs the machine gets stuck, and a dependent type theory needs a name, a *neutral term*, for every stuck computation. A stuck pure computation names itself: Lean’s $\text{Nat.add } n \ 0$ is its own normal form. A stuck effectful call does not, because part of what it produces lives in the places it borrowed. We *close off* such a call: its result and the final content of each borrowed place are replaced by *sealed programs*, closed source programs that own their state. For AddM on an unknown σ , the borrowed place receives $[\text{let } c = \sigma; \text{AddM } \&c \ 0; c]$, which is the pure wrapper a programmer would write anyway. When a later case split refines σ , the sealed program runs again. Sealed programs are exactly Aeneas’s backward functions, but they are written in the source language and generated on demand, so the programmer never writes or reads a second language. A function that returns a borrow is closed off into a sealed program with a hole awaiting the final value of the returned borrow; this is Aeneas’s region abstraction, in the same form.

The environment does the congruence. When the checker types the recursive call $\text{AddMZero } \&p$, it evaluates the callee’s statement in the caller’s environment, where $\&p$ points into the predecessor field of the caller’s number. The observation therefore sees the successor that surrounds the recursive call, and the induction hypothesis arrives already wrapped in S : it is literally the goal. The step $\text{cong } S \ i h$ that a pure proof writes by hand has been performed by the borrow structure. Soundness rests on a frame property that Rust’s ownership discipline provides: a call can affect only what it is passed.

1.3 Why this is not impossible

Pédrot and Tabareau’s fire triangle [16] shows that a type theory cannot have substitution of arbitrary terms, dependent elimination and observable effects at once without becoming inconsistent. Ochr is call-by-value, keeps dependent elimination, and restricts what may be substituted into a type: a type only ever contains values. A computation mentioned in a type is run on a private copy of the environment, and only its observation, a value, enters the type. Ownership is what makes this restriction cheap. A closed Ochr program can mutate only places it created, so every closed program behaves like a value, and the observation of an open program is a value of a known shape.

1.4 Contributions

- A core calculus, *Ochr*, combining mutable borrows with a dependent type theory with a universe of proof-irrelevant propositions, whose definitional equality unfolds imperative code (Section 3, Section 4).
- *Closing off*, which gives every stuck effectful call a neutral form made of sealed source programs, including calls that return borrows (Section 4).
- An observational equality between computations that is derived rather than primitive (Section 5).
- Worked examples, among them the equivalence of **AddM** with a version that first obtains a borrow of the final node and then writes through it, each proved by bare structural recursion (Section 2).
- A translation into the calculus of inductive constructions, in which the backward functions reappear, establishing consistency, and an adequacy theorem relating symbolic and concrete evaluation (Section 7).
- An executable implementation of the checker in Lean 4 that checks every example in the paper (Section 8).

2 Ochr by example

This section introduces *Ochr* through its running examples. Everything is informal here; Section 3 onwards makes it precise.

2.1 Imperative code is evaluated inside types

Ochr programs are made of definitions with parameters and a body. Borrow types $\&T$ are Rust’s mutable references; places are variables x , dereferences $*p$ and the predecessor field $p.1$ of a number. A **match** on a place binds its pattern variable to a *sub-place*, not a copy, so in **match** $*x \{ Z \Rightarrow \dots \mid S \ p \Rightarrow \dots \}$ the name p stands for the field $(*x).1$ and $\&p$ reborrows it.

Add wraps **AddM** behind a pure interface. It owns its first argument, lends it to **AddM**, and returns it:

```
Add(x : Nat, y : Nat) : Nat := AddM(&x, y); x
```

Because definitional equality is evaluation, $\text{Id Nat (Add(2, 3)) 5}$ holds by **refl**: the checker runs **Add**(2, 3), which runs **AddM** in place on a local copy, and compares the result with 5. There is nothing special about imperative code here, and nothing special about **Add** being “pure”: it is simply a program whose observable output is its result.

The machine that does this is the symbolic semantics of Aeneas’s low-level borrow calculus [7]. An environment Ω maps variables to values; borrowing x moves its content into the borrow and leaves a *loan* behind; ending the borrow moves the content back. Running **Add**(2, 3) passes through these states:

```
{ x ↦ 2 }                               // enter Add
{ x ↦ loan₀ } ⊢ AddM(borrow₀ 2, 3)       // &x: the content moves into the borrow
{ x ↦ loan₀ | x' ↦ borrow₀ (S loan₁), ... } // AddM matched S and reborrowed the field
...
{ x ↦ 5 }                               // borrows ended, contents returned
```

2.2 Symbolic inputs, and what a stuck call leaves behind

To reason about all numbers at once, the checker runs programs on *abstract values* σ , which play the role of Lean’s free variables. Running **Add**(σ , 0) borrows $x \mapsto \sigma$ and calls **AddM**(**borrow**₀ σ , 0), whose body immediately gets stuck: it must match on σ .

In a pure type theory a stuck term is its own normal form. Here the call has also produced something that lives in the environment, namely the final content of the place it borrowed, and that content needs a name. We *close off* the call: the call returns, and the loan it held is filled with a *sealed program*, a closed source program that owns its state and computes that content:

```
{ x ↦ loan₀ } ⊢ AddM(borrow₀ σ, 0)
→ { x ↦ [let c = σ; AddM(&c, 0); c] } ⊢ ()
```

Write $N(\sigma)$ for $[let\ c = \sigma; AddM(\&c, 0); c]$. It is a neutral value, like a stuck `Nat.rec` in Lean, and it is literally the body of `Add(σ, 0)`: the sealed program *is* the pure wrapper, generated on demand. It is also exactly the value that Aeneas’s backward function for `AddM` would compute [7], written without leaving the source language. When a later case split refines σ to $S\ \sigma'$, $N(S\ \sigma')$ runs again, now makes progress through one successor, closes off the inner recursive call, and normalises to $S\ N(\sigma')$.

2.3 Equality between computations

The type `Id Unit (AddM(x, 0)) ()` compares two computations. It is evaluated at the environment where it is formed. Each side runs on its own copy of that environment; the result and the final contents of the places the side may write form its *observation*; and `Id` computes to ordinary equality between the two observations. For a definition taking $x : \&Nat$, the checker works at the definition’s *generic call* `AddMZero(&c)` from $\{ c \mapsto \sigma \}$, so the place that may be written is `c`, the owner of x ’s loan:

```
[[AddM(x, 0)]] = ((), N(σ))      // the call is stuck, so c receives the sealed program
[()]          = ((), σ)         // nothing happens; ending x's borrow returns σ to c

Id Unit (AddM(x, 0)) () ≡ Eq (Unit × Nat) ((), N(σ)) ((), σ) ≡ Eq Nat N(σ) σ
```

The last step uses the observational computation rules for equality [3]: equality of pairs is a conjunction, reflexive equations are τ , and τ is a unit for $\wedge$. The statement “running `AddM(x, 0)` has no effect” has become the ordinary proposition that $N(\sigma)$ equals σ .

2.4 The environment performs the congruence

Now the proof:

```
AddMZero(x : &Nat) : Id Unit (AddM(x, 0)) () :=
  match *x { Z => refl | S p => AddMZero(&p) }
```

The match on `*x` splits on σ . In the `Z` branch the goal is `Eq Nat N(Z) Z`; $N(Z)$ runs to `Z`, the equation is reflexive and so τ , and `refl` proves it. In the `S` branch the goal is `Eq Nat N(S σ') (S σ')`, which normalises to `Eq Nat (S N(σ')) (S σ')`.

The recursive call is checked by the same rule that produced the goal: the callee’s statement is evaluated at the call site, with its parameter bound to the argument. At the call site the environment is

```
{ c ↦ loan₀ | x ↦ borrow₀ (S loan₁) } and the argument is borrow₁ σ'
```

because matching `S p` through the borrow and taking `&p` leaves the successor in place, around a loan for the predecessor. The owner of the argument’s loan is found by following it outwards: `loan₁` sits inside x ’s borrow, whose loan sits in `c`. So the recursive call’s statement observes `c`, and running `AddM` on the argument fills `loan₁` with $N(\sigma')$, after which `c` holds $S\ N(\sigma')$:

```
[[AddM(x', 0)]] at the call site = ((), S N(σ'))
[()]           at the call site = ((), S σ')
AddMZero(&p) : Eq Nat (S N(σ')) (S σ')
```

This is the goal, symbol for symbol. The successor that a pure proof would add with `cong S` was supplied by the environment. The recursive call is a proof, and proofs are erased at runtime, so the checker does not run it: its borrow argument is returned unchanged.

For comparison, here is the same theorem about the pure wrapper:

```
AddZero(x : Nat) : Id Nat (Add(x, 0)) x :=
  match x { Z => refl | S p => cong S (AddZero(p)) }
```

$\text{Add}(x, 0)$ writes nothing outside itself, so its observation is just its result, and the goal is again $\text{Eq Nat } N(\sigma) \sigma$, the very proposition AddMZero 's statement computed to. The induction hypothesis $\text{AddZero}(p)$ is about a fresh copy of the predecessor, so the successor must be added by hand. In Ochr the in-place proof is the shorter one. The two theorems are also interchangeable: $\text{AddZero}(x) := \text{AddMZero}(\&x)$ type-checks, because both statements normalise to the same proposition.

2.5 Returning a borrow

A more idiomatic in-place addition first finds the final node and then writes through it:

```
TailM(x : &Nat) : &Nat := match *x { Z => x | S p => TailM(&p) }
AddM'(x : &Nat, y : Nat) : Unit := let t = TailM(x); *t := y
```

TailM returns a borrow into its argument. When it is stuck on σ , the place it borrowed cannot be given its final content yet, because that depends on what the caller will later write through the returned borrow. Closing off such a call returns a borrow of a fresh k and fills the argument's loan with a sealed program that has a *hole* loan_k for the returned borrow's final content:

```
{ c ↦ loanσ } ⊢ TailM(borrowσ σ)
  → { c ↦ [let c = σ; let r = TailM(&c); *r := loank; c] } ⊢ borrowk [let c = σ; let r
  = TailM(&c); *r]
```

In AddM' , the caller writes y through t and drops it; ending borrow_k substitutes the final content y for the hole. What remains in c is $[\text{let } c = \sigma; \text{let } r = \text{TailM}(\&c); *r := y; c]$: the effect of AddM' , as a program. The hole is the neutral form of an Aeneas region abstraction, and of a RustHorn prophecy [13]: it records only the *final* value written through the returned borrow.

The equivalence of the two additions is proved by the same bare recursion:

```
AddMEq(x : &Nat, y : Nat) : Id Unit (AddM(x, y)) (AddM'(x, y)) :=
  match *x { Z => refl | S p => AddMEq(&p, y) }
```

In the successor branch, re-running AddM' 's sealed program on $S \sigma'$ unfolds TailM once, closes off its inner call with a fresh hole, and the pending write $*r := y$ then fills that hole; on the other side the recursive call's statement, evaluated at the call site, performs the same steps through the caller's borrow. Both sides arrive at $\text{Eq Nat } (S A(\sigma', y)) (S B(\sigma', y))$, where A and B are the sealed programs for the two additions on the predecessor. The theorem for an owned number follows by lending it: $\text{AddMEqOwned}(x : \text{Nat}) : \text{Id Unit } (\text{AddM}(\&x, 0)) (\text{AddM}'(\&x, 0)) := \text{AddMEq}(\&x, 0)$.

2.6 Branching

A `match` on an abstract value in the middle of a function cannot pick an arm. Each arm is checked separately, and the rest of the function is then checked once, from the state in which the match itself has been closed off like a call. This covers borrows whose origin depends on the branch:

```
AddToOne(b : Nat, x1 : &Nat, x2 : &Nat, y : Nat) : Unit :=
  let r = match b { Z => x1 | S _ => x2 }; AddM(r, y)
```

The closed-off match returns a borrow with a hole that appears in the sealed programs for both x_1 's and x_2 's places; whichever the match would have chosen receives the final content. A proof about AddToOne splits on b , after which the sealed programs run and the goal becomes a statement about AddM alone.

3 The calculus

Draft pending rule set v1.

4 Evaluation and closing off

Draft pending rule set v1.

5 Observational equality

Draft pending rule set v1.

6 Typing

Draft pending rule set v1.

7 Metatheory

Draft pending the model.

8 Implementation

Draft pending the Lean checker.

9 Related work

Functional translation of borrows. Aeneas [7] translates safe Rust into a pure functional program in Lean, F* or Coq. Its symbolic semantics of the low-level borrow calculus is the direct ancestor of our machine: we take its values with loans and borrows, its lazy ending of borrows, and its insight that a mutable borrow parameter is a forward computation plus a backward function. Later work gives that symbolic semantics joins and loops and proves it sound as a borrow checker [8]. The difference is where and when the translation happens. Aeneas translates eagerly, whole-program, once, and the user reasons about the translated artifact, where backward functions are named definitions. Ochr never produces a translated program. The same state-passing content appears lazily inside conversion, only as far as a particular equation needs, and only in source syntax: a sealed program is what Aeneas would call an application of a backward function. Aeneas is modular in that it never unfolds a callee; Ochr must unfold, because unfolding is what definitional equality is, and closing off is what keeps the unfolding finite.

Prophecies. RustHorn [13], RustHornBelt [12] and Creusot [5] model a mutable borrow as its current value paired with a *prophecy* $\wedge x$ of its final value, resolved when the borrow ends. A filled loan in Ochr is a prophecy that has been resolved by the machine, and the hole in a sealed program for a returned borrow is an unresolved one. Prophecies are logic variables introduced at borrow creation; Ochr introduces no variables at all outside binders, which is what keeps concrete evaluation free of symbols (§Section 7).

Specification-based verification of imperative code. Verus [9], Creusot, Prusti, Flux [10] and VeriFast verify imperative code against specifications written separately, in pre- and postconditions, refinement types or separation logic, and discharge the obligations with automated provers. Hoare type theory [14] and Ynot internalise the triples in a dependent type theory; F*’s Dijkstra monads [1] internalise weakest preconditions and let **reify** turn a stateful computation into a state-passing function. In all of these the effectful program is specified, not compared: two programs with the same specification are not thereby equal. In Ochr the specification

of an in-place function is another program, or a theorem about the function itself, and equality of computations is a type.

Effects in dependent type theory. The fire triangle [16] shows that substitution, dependent elimination and observable effects cannot coexist consistently, and ∂ CBPV organises the possible compromises. Ochr sits in the call-by-value corner: dependent elimination is kept, and substitution into types is restricted to values, where a computation in a type contributes only its observation. The observation of a computation is *thunkable* in their sense, and ownership gives a syntactic criterion for thunkability, which they note is otherwise hard to capture intensionally. Ahman’s eMLTT [2] equips computations with definitional equations from an algebraic theory of global state; Ochr’s state is structured, local and borrow-shaped, and its equations come from running the program rather than from an axiomatised theory.

Observational equality. Observational type theory [3, 15] computes the equality type by the structure of the type: equality of pairs is a pair of equalities, equality of functions is pointwise. Ochr’s **Id** between computations is an instance of this idea at the type of observations, and we adopt the observational computation rules for the data types of the core.

Pure languages with in-place updates. Lean 4 performs destructive updates on values it can prove unshared at runtime [17], and Koka’s fully in-place fragment [11] guarantees statically that a function allocates nothing. Programs in these languages are pure, so their proofs are ordinary, but the programmer writes functional code and cannot express a borrow that is returned to the caller. Linear and quantitative type theories [4] and ATS [18] track resources in types; Ochr uses the same separation between runtime usage and erased type-level usage as quantitative type theory, where a variable used in a type is not consumed.

Relational reasoning. ReLoC [6] and relational Hoare logics prove two programs equivalent inside a program logic. Ochr’s **Id** is a relational statement about two computations, but it is decided, where possible, by conversion, and it lives in the same type theory as the programs.

10 Conclusion

Draft.

Bibliography

- [1] Danel Ahman, Catalin Hritcu, Kenji Maillard, Guido Martínez, Gordon Plotkin, Jonathan Protzenko, Aseem Rastogi, and Nikhil Swamy. 2017. Dijkstra Monads for Free. In *POPL*, 2017.
- [2] Danel Ahman. 2018. Handling Fibred Algebraic Effects. In *Proc. ACM Program. Lang. (POPL)*, 2018.
- [3] Thorsten Altenkirch, Conor McBride, and Wouter Swierstra. 2007. Observational Equality, Now!. In *PLPV*, 2007.
- [4] Robert Atkey. 2018. Syntax and Semantics of Quantitative Type Theory. In *LICS*, 2018.
- [5] Xavier Denis, Jacques-Henri Jourdan, and Claude Marché. 2022. Creusot: A Foundry for the Deductive Verification of Rust Programs. In *ICFEM*, 2022.
- [6] Dan Frumin, Robbert Krebbers, and Lars Birkedal. 2018. ReLoC: A Mechanised Relational Logic for Fine-Grained Concurrency. In *LICS*, 2018.
- [7] Son Ho and Jonathan Protzenko. 2022. Aeneas: Rust Verification by Functional Translation. In *Proc. ACM Program. Lang. (ICFP)*, 2022. 116:1–116:31.
- [8] Son Ho, Aymeric Fromherz, and Jonathan Protzenko. 2024. Sound Borrow-Checking for Rust via Symbolic Semantics. In *Proc. ACM Program. Lang. (ICFP)*, 2024.
- [9] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. 2023. Verus: Verifying Rust Programs using Linear Ghost Types. In *Proc. ACM Program. Lang. (OOPSLA)*, 2023.

- [10] Nico Lehmann, Adam T. Geller, Niki Vazou, and Ranjit Jhala. 2023. Flux: Liquid Types for Rust. In *PLDI*, 2023.
- [11] Anton Lorenzen, Daan Leijen, and Wouter Swierstra. 2023. FP^2 : Fully in-Place Functional Programming. In *Proc. ACM Program. Lang. (ICFP)*, 2023.
- [12] Yusuke Matsushita, Xavier Denis, Jacques-Henri Jourdan, and Derek Dreyer. 2022. RustHornBelt: A Semantic Foundation for Functional Verification of Rust Programs with Unsafe Code. In *PLDI*, 2022.
- [13] Yusuke Matsushita, Takeshi Tsukada, and Naoki Kobayashi. 2020. RustHorn: CHC-based Verification for Rust Programs. In *ESOP*, 2020.
- [14] Aleksandar Nanevski, Greg Morrisett, and Lars Birkedal. 2006. Polymorphism and Separation in Hoare Type Theory. In *ICFP*, 2006.
- [15] Loïc Pujet and Nicolas Tabareau. 2022. Observational Equality: Now for Good. *Proc. ACM Program. Lang. (POPL)* 6, (2022).
- [16] Pierre-Marie Pédrot and Nicolas Tabareau. 2020. The Fire Triangle: How to Mix Substitution, Dependent Elimination, and Effects. *Proc. ACM Program. Lang. (POPL)* 4, (2020), 58:1–58:28.
- [17] Sebastian Ullrich and Leonardo de Moura. 2019. Counting Immutable Beans: Reference Counting Optimized for Purely Functional Programming. In *IFL*, 2019.
- [18] Hongwei Xi. 2003. Applied Type System (extended abstract). In *TYPES*, 2003.